

The Fake-Busy and True-Idle Problems of Running Graph Applications on Chiplet-Based Multi-cores

Rashid Aligholipour
rashid.aligholipour@it.uu.se
Uppsala University, Sweden

Yuan Yao
yuan.yao@it.uu.se
Uppsala University, Sweden

Abstract—We introduce the **fake-busy** and **true-idle** problems encountered when running large graph workloads on chiplet-based Out-of-Order (OoO) multi-cores. Caused by high inter-chiplet communication latency and irregular memory access patterns, these issues lead to inefficient use of core pipeline resources such as the Reorder Buffer (RoB) and Load Queue (LQ). Our evaluation shows that reducing RoB and LQ sizes has minimal impact on application performance, revealing new performance optimization opportunities for large graph workloads.

I. INTRODUCTION

The computational demands of graph applications have grown significantly in recent years [1]–[6], posing substantial challenges for conventional computing systems. To address these challenges, domain-specific architectures, including graph-specific accelerators [7]–[9] and processing-in-memory (PIM)-based graph acceleration [10], [11], have been developed. Recently, advances in chiplet-based architectures [12]–[17] have made it feasible to efficiently process large-scale graphs on general-purpose multi-cores, providing a promising alternative for scalable graph processing.

A primary challenge in chiplet-based multi-cores is the high latency associated with off-chiplet data communication [18], [19]. As illustrated in Figure 1, in a chiplet-based multi-core, core-dies are typically interconnected via a memory and I/O die (M/I/O-die). Each core-die hosts local Out-of-Order (OoO) cores (with private cache) and a locally shared last-level-cache (LLC) for the cores within the core-die. The M/I/O-die serves as a central hub, connecting each core-die through an inter-socket (IS) interface and providing a set of globally shared memory controllers (MCs), which are connected to off-chip DRAM banks. Within the M/I/O-die, data are routed through a local interconnect network consisting of data switches (DS). In this configuration, a miss in a core-die's local LLC (①) must first traverse the IS interface to the M/I/O-die (②), then routed by the data switch to the corresponding memory controller (③) that is responsible for the corresponding off-chip DRAM rank. As such, this multi-step off-chiplet data transfer introduces significant latency for misses in the LLCs.

Key Findings. In this paper, we study the micro-architectural components, such as the Reorder Buffer (ROB) and Load Queue (LQ), of OoO cores when running graph applications on chiplet-based multi-core systems. We observe that the large and irregular memory footprints of graphs often lead to inaccurate branch predictions and unsuccessful/

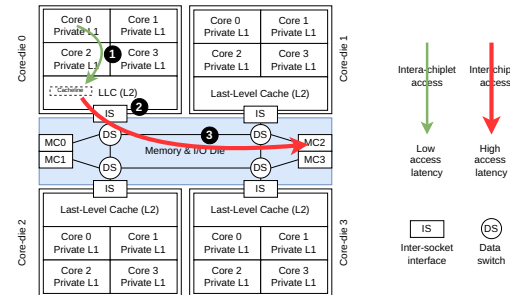


Fig. 1: Chiplet-based multi-core architecture.

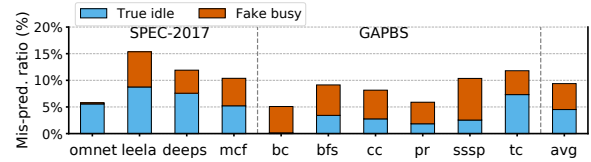


Fig. 2: Branch miss-prediction ratio.

ful/squashed speculative executions, resulting in significant wastage of core pipeline resources and increased power consumption without contributing meaningfully to application execution time. Specifically, our experiments reveal that running graph applications on chiplet-based OoO multi-cores results in two key challenges: **fake-busy** and **true-idle**.

In **fake-busy**, while a core waits for a branch-resolving load, it speculatively *skips* branch execution, advances to the next loop iteration, and issues new speculative branch-loads for future iterations, which can recursively trigger additional branch-loads. Due to the long latency of off-chiplet data transfers, while waiting for the initial branch to be resolved, **fake-busy** can quickly fill the ROB and LQ to their full capacities, ultimately causing the entire core pipeline to stall.

In **true-idle**, while waiting for the leading branch-load, a core speculatively executes code *within* the branch, which often implements a certain graph algorithms, such as breadth-first search (BFS), *etc.* However, during speculative execution, instructions within the branch can be fetched, decoded, and renamed, but cannot be actually executed as they are waiting for the missing branch-load. Consequently, **true-idle** results in that instructions are stalled in the ROB, leading to an underutilized core pipeline.

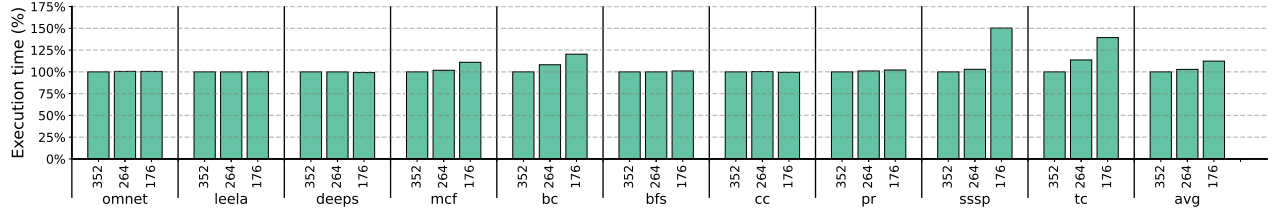


Fig. 3: Application execution time with varied ROB sizes.



Fig. 4: Application execution time with varied LQ sizes.

II. METHODOLOGY

We simulated a system as depicted in Figure 1 with system-emulation (SE) simulations in gem5 [20], using *x86_64 OoO* cores and the *Ruby* memory model. Additionally, we leverage gem5’s HetroGarnet [21] to simulate inter-socket (IS) interface and data switches (DS) for the I/O die, ensuring to capture the cross-V/F-domain overheads between chiplets and the I/O die. We use McPAT [22], which is based on CACTI [23], for reporting power consumption-related results. Our benchmark suite includes a diverse set of graph benchmarks with varying characteristics. We utilize four SPEC CPU2017 [24] programs—*omnetpp*, *leela*, *deepsjeng*, and *mcf*. In addition, we employ six programs—*bc*, *bfs*, *cc*, *pr*, *sssp*, and *tc*—from the GAP benchmark suite [25].

Figure 2 illustrates the branch mis-prediction ratio, *i.e.*, the number of *incorrectly* predicted but committed conditional-branches (c-branches) over *all* committed c-branches. Additionally, the results are categorized into *fake-busy* (branches predicted as not taken but actually taken) and *true-idle* (branches predicted as taken but actually not taken).

The figure shows that 1) branch mis-predictions occur across all benchmarks, with an average ratio of 9.06%, *i.e.*, a mis-prediction occurs roughly once every 10 committed c-branches. 2) The ratios are relatively low in *omnetpp*, *bc*, and *pr*, as our baseline TAGE [26] branch predictor is effective for these benchmarks due to their periodic or semi-regular memory access patterns (*e.g.*, repeated iterative updates to node ranks in *pr*). 3) Both *fake-busy* and *true-idle* occur in most benchmarks (except for *omnetpp* and *bc*, which are dominated by *true-idle* and *fake-busy*, respectively). This is because *fake-busy* and *true-idle* are *mutualistic*, *i.e.*, they are often linked to different paths of the same branch during graph traversal. This mutualistic relationship leads applications to alternately incur *fake-busy* and *true-idle* while traversing different regions of a graph. 4) While both *fake-busy* and *true-idle* occur, *true-idle* is more prevalent in *leela*, *deepsjeng*,

mcf, and *tc*, whereas *fake-busy* is more prevalent in *bfs*, *cc*, *pr*, and *sssp*.

Figure 3 illustrates the application execution time for varying ROB sizes: 50% capacity (176 entries), 75% capacity (264 entries), and 100% capacity (352 entries, our baseline). On average, a 75%-size ROB increases application runtime by 2.8%, while a 50%-size ROB results in an increase of 12.3% (4.3% when excluding *sssp* and *tc*, whose execution times increase significantly with a 50% reduction in ROB size). The results demonstrate that due to frequent branch misprediction and instruction squashing in the ROB, reducing the ROB size has a limited impact on application execution time. Figure 4 presents the execution time of applications under different Load Queue (LQ) sizes: 50% (64 entries), 75% (96 entries), and 100% (128 entries). As observed, reduced LQ sizes have minimal impact on execution time too, which can be attributed to most loads in the LQ are squashed due to branch misprediction.

III. CONCLUSION

This paper identifies two key challenges when running graph applications on chiplet-based out-of-order (OoO) multi-cores: *fake-busy*—where a core skips unresolved branches and issues speculative loads for future iterations—and *true-idle*—where execution stalls within unresolved branches. Both result in mis-speculation, leading to squashed instructions and increased power consumption. Sensitivity studies show that reducing ROB/LQ sizes can preserve application performance, offering practical insights for performance optimization in running big graph applications on chiplet-based systems.

ACKNOWLEDGMENTS

We acknowledge the use of computing resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS) under project numbers NAISS 2023/22-1215, NAISS 2024/5-314, and NAISS 2024/6-189.

REFERENCES

- [1] H. Kwak, C. Lee, H. Park, and S. Moon, "What is Twitter, A Social Network or A News Media?" in *International Conference on World Wide Web*, 2010, p. 591–600.
- [2] Graph-Powered Machine Learning at Google. [Online]. Available: <https://research.google/blog/graph-powered-machine-learning-at-google/>
- [3] T. D. Bui, S. Ravi, and V. Ramavajjala, "Neural Graph Learning: Training Neural Networks Using Graphs," in *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining (WSDM)*, 2018, p. 64–71.
- [4] A. Outman, "Web Data Commons - Hyperlink Graphs," 2017. [Online]. Available: <https://dx.doi.org/10.21227/H23S3B>
- [5] X. Zeng, X. Song, T. Ma, X. Pan, Y. Zhou, Y. Hou, Z. Zhang, K. Li, G. Karypis, and F. Cheng, "Repurpose Open Data to Discover Therapeutics for COVID-19 Using Deep Learning," *Journal of Proteome Research*, vol. 19, no. 11, pp. 4624–4636, 2020.
- [6] V. Gligorić, P. D. Renfrew, T. Kosciół, J. K. Leman, D. Berenberg, T. Vatanen, C. Chandler, B. C. Taylor, I. M. Fisk, H. Vlamakis, R. J. Xavier, R. Knight, K. Cho, and R. Bonneau, "Structure-Based Protein Function Prediction Using Graph Convolutional Networks," *Nature Communications*, vol. 12, no. 1, p. 3168, May 2021.
- [7] T. J. Ham, L. Wu, N. Sundaram, N. Satish, and M. Martonosi, "Graphicionado: A High-Performance and Energy-Efficient Accelerator for Graph Analytics," in *International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–13.
- [8] J. Zhao, Y. Zhang, J. Cheng, Y. Wu, C. Ye, H. Yu, Z. Huang, H. Jin, X. Liao, L. Gu, and H. Liu, "SaGraph: A Similarity-Aware Hardware Accelerator for Temporal Graph Processing," in *Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [9] C. Gao, M. Afarin, S. Rahman, N. Abu-Ghazaleh, and R. Gupta, "MEGA Evolving Graph Accelerator," in *International Symposium on Microarchitecture (MICRO)*, 2023, pp. 310–323.
- [10] C. Shin, T. Kwon, J. Song, J. H. Ju, F. Liu, Y. Choi, and J. Lee, "A Case for In-Memory Random Scatter-Gather for Fast Graph Processing," *IEEE Computer Architecture Letters (CAL)*, vol. 23, no. 1, pp. 73–77, 2024.
- [11] X. Li, R. Ausavarungrun, X. Liu, X. Liu, X. Zhang, H. Lu, Z. Song, N. Jing, and X. Liang, "Gzippo: Highly-compact Processing-In-Memory Graph Accelerator Alleviating Sparsity and Redundancy," in *International Conference On Computer Aided Design (ICCAD)*, 2022, pp. 1–9.
- [12] S. Naffziger, N. Beck, T. Burd, K. Lepak, G. H. Loh, M. Subramony, and S. White, "Pioneering Chiplet Technology and Design for the AMD EPYC™ and Ryzen™ Processor Families : Industrial Product," in *International Symposium on Computer Architecture (ISCA)*, 2021, pp. 57–70.
- [13] A. Sangiovanni-Vincentelli, Z. Liang, Z. Zhou, and J. Zhang, "Automated Design of Chiplets," in *International Symposium on Physical Design (ISPD)*, 2023, pp. 1–8.
- [14] G. Krishnan, A. A. Goksoy, S. K. Mandal, Z. Wang, C. Chakrabarti, J.-s. Seo, U. Y. Ogras, and Y. Cao, "Big-Little Chiplets for In-Memory Acceleration of DNNs: A Scalable Heterogeneous Architecture," in *International Conference On Computer Aided Design (ICCAD)*, 2022, pp. 1–9.
- [15] G. H. Loh and R. Swaminathan, "The Next Era for Chiplet Innovation," in *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2023, pp. 1–6.
- [16] M. Evers, L. Barnes, and M. Clark, "The AMD Next-Generation Zen 3 Core," *IEEE Micro*, vol. 42, no. 3, pp. 7–12, 2022.
- [17] AMD Infinity Architecture. [Online]. Available: <https://www.amd.com/system/files/documents/LE-70001-SB-InfinityArchitecture.pdf>
- [18] P. Coudrain, J. Charbonnier, A. Garnier, P. Vivet, R. Vélard, A. Vinci, F. Ponthenier, A. Farcy, R. Segaud, P. Chausse, L. Arnaud, D. Latard, E. Guthmuller, G. Romano, A. Gueugnot, F. Berger, J. Beltritti, T. Mourier, M. Gottardi, S. Minoret, C. Ribière, G. Romero, P.-E. Philip, Y. Exbrayat, D. Scevola, D. Campos, M. Argoud, N. Allouti, R. Eleouet, C. Fuguet Tortolero, C. Aumont, D. Dutoit, C. Legalland, J. Michailos, S. Chéramy, and G. Simon, "Active Interposer Technology for Chiplet-Based Advanced 3D System Architectures," in *Electronic Components and Technology Conference (ECTC)*, 2019, pp. 569–578.
- [19] N. Kim, D. Wu, D. Kim, A. Rahman, and P. Wu, "Interposer Design Optimization for High Frequency Signal Transmission in Passive and Active Interposer Using Through Silicon Via (TSV)," in *Electronic Components and Technology Conference (ECTC)*, 2011, pp. 1160–1167.
- [20] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 Simulator," *SIGARCH Computer Arch. News*, vol. 39, no. 2, pp. 1–7, aug 2011.
- [21] S. Bharadwaj, J. Yin, B. Beckmann, and T. Krishna, "Kite: A Family of Heterogeneous Interposer Topologies Enabled via Accurate Interconnect Modeling," in *Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [22] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "McPAT: An Integrated Power, Area, and Timing Modeling Framework for Multicore and Manycore Architectures," in *International Symposium on Microarchitecture (MICRO)*, 2009, pp. 469–480.
- [23] R. Balasubramanian, A. B. Kahng, N. Muralimanoohar, A. Shafiee, and V. Srinivas, "CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories," *Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 14:1–14:25, 2017.
- [24] "SPEC's Benchmarks and Tools." [Online]. Available: <https://www.spec.org/>
- [25] S. Beamer, K. Asanović, and D. Patterson, "The GAP Benchmark Suite," 2015. [Online]. Available: <https://arxiv.org/abs/1508.03619>
- [26] A. Seznec, "A New Case for the TAGE Branch Predictor," in *International Symposium on Microarchitecture (MICRO)*, ser. MICRO-44. New York, NY, USA: Association for Computing Machinery, 2011, p. 117–127. [Online]. Available: <https://doi.org/10.1145/2155620.2155635>